



Programação Servidor com JAVA

A criação de sistemas Java para o ambiente Web, com descrição do ambiente de execução e construção de exemplos de sistemas com tecnologias Servlet, JSP e acesso a banco de dados.

Prof. Denis Gonçalves Cople | Colaboração Prof. Kleber de Aguiar

Propósito

Elaborar sistemas para o ambiente Web, com utilização de tecnologia Java, e definir soluções que funcionem através do protocolo HTTP, com o acesso a partir de navegadores ou plataformas móveis, indo ao encontro das demandas de um mercado em que a conectividade se tornou uma necessidade primária.

Preparação

Antes de iniciar o conteúdo deste tema, é necessário configurar o ambiente, com a instalação do JDK e Apache NetBeans, definindo a plataforma de desenvolvimento. Também é necessário instalar o *Web Server* Tomcat e o *Application Server* GlassFish, definindo o ambiente de execução e testes. Sugere-se configurar a porta do servidor Tomcat como 8084 para evitar conflitos com o GlassFish na porta 8080.

Objetivos

- Identificar as características de Web Servers no ambiente Java.
- Identificar as características de Application Servers no ambiente Java.
- Empregar as tecnologias Servlet e JSP na construção de aplicativos servidores.
- Empregar a tecnologia JDBC para viabilizar o acesso ao banco de dados.

Introdução

No contexto do ambiente de desenvolvimento de sistemas Web, vamos analisar os princípios funcionais de sistemas cliente-servidor, no modelo Web, e veremos como são configurados os servidores Tomcat e GlassFish, bem como os aplicativos que executam nas duas plataformas.

Após compreender o ambiente de execução, iremos estudar os componentes Servlet e JSP, definindo aplicativos servidores, e adicionaremos o acesso ao banco de dados nos sistemas criados, com o uso de JDBC (Java Database Connectivity).

O protocolo HTTP e o ambiente servidor

Protocolo HTTP

Neste vídeo, apresentaremos o protocolo HTTP. Vamos lá?

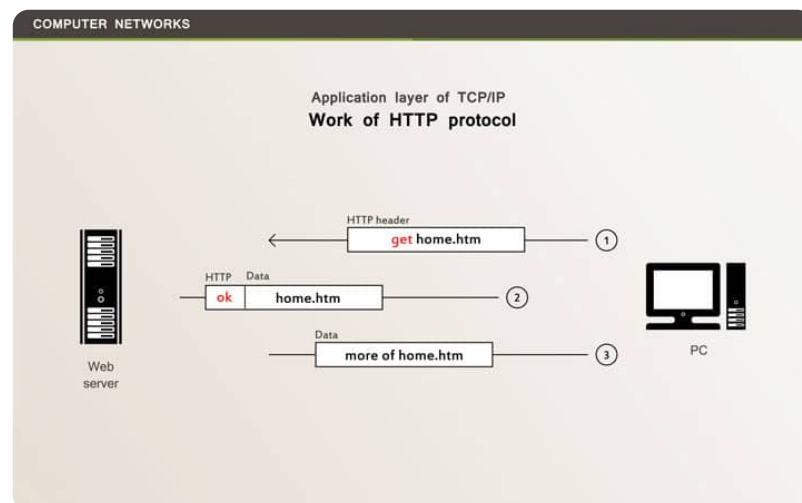


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

O protocolo **HTTP** (Hypertext Transfer Protocol) pertence à camada de aplicação do modelo **OSI** (Open System Interconnection), definido originalmente para suportar páginas de hipertexto baseadas na sintaxe **HTML** (Hypertext Markup Language).

As informações transitam de diversas formas no protocolo HTTP, definidas a partir da escolha do método de transmissão utilizado. Os dois métodos mais comuns são **GET**, por meio do qual a informação é transmitida junto ao endereço, e **POST**, com a informação enviada para o servidor em *background*, de forma que não fique exposta para o usuário.



Modo de trabalho do protocolo HTTP.

Considerando o objetivo inicial, tudo que tínhamos eram conjuntos de páginas com conteúdo estático e a possibilidade de navegar entre elas a partir de *hyperlinks*, logo, o máximo de dinamismo possível era a execução de rotinas em JavaScript, ou a obtenção de dados via Sockets, com a utilização de Applets Java.



Comentário

Com a evolução da internet, as necessidades mudaram, e o processamento no servidor levou a um novo patamar na criação de sistemas para Web. Passamos a gerar respostas dinamicamente, com a possibilidade de utilização dos dados disponíveis no ambiente servidor, como tabelas dos bancos de dados, repositórios de arquivos, ou até mesmo informações obtidas por canais de comunicação com outras plataformas.

Como as funcionalidades foram ampliadas, o perfil dos dados utilizados na comunicação também se diversificou. Atualmente, temos a adoção de **XML** (Extended Markup Language) e **JSON** (Java Script Object Notation), entre outros formatos, utilizados principalmente na representação de dados, como em operações cadastrais ou financeiras.



Atenção

De acordo com a metodologia mais aceita, atualmente, no que se refere à construção de páginas, a estruturação do conteúdo utiliza HTML, dados transitam nos formatos XML ou JSON, e a formatação tipográfica é baseada em CSS (Cascading Style Sheets).

Como vamos gerar o conteúdo a partir de processamento efetuado no servidor, as respostas deverão ser constituídas com o uso dessas sintaxes e criadas por meio da linguagem de programação utilizada pelo servidor, como: **Java**, **Perl**, **PHP** ou **C#**.

Ambiente servidor para web

Neste vídeo, iremos nos aprofundar no ambiente servidor para web. Vamos lá?



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Quando enviamos os dados para alguma tecnologia servidora, temos uma requisição **HTTP**, que poderá iniciar algum processamento no servidor e que, ao final, retornará uma resposta, normalmente com conteúdo **HTML** ou **XML**. Dentro do modelo de requisição e resposta, proporcionado pelo HTTP, o conteúdo recebido constitui um ambiente independente do anterior, no qual ocorreu a chamada, ou seja:

É um protocolo que não oferece a manutenção de estados.

Se utilizássemos um servidor HTTP simples, teríamos apenas um repositório de páginas e recursos de mídia, de forma geral, com todo o processamento ocorrendo no cliente, mas servidores como Apache **Tomcat** e **IIS** (Internet Information Services) irão permitir o processamento no servidor. Em geral, temos a requisição do cliente, a interceptação do servidor, a captura de possíveis dados enviados, a execução de alguma rotina na linguagem adotada, a construção da resposta e o retorno para o cliente.

Utilizaremos, no ambiente Java, uma implementação da interface **HttpServletRequest**, normalmente denominada **request**, para capturar os dados enviados pela requisição HTTP, efetuando qualquer tipo de processamento com esses dados, como a inserção em banco de dados ou as validações de segurança. A resposta, por sua vez, é encapsulada em uma implementação de **HttpServletResponse**, frequentemente com o nome **response**, que deverá emitir o conteúdo gerado através do canal de saída padrão da conexão.

```
java

int a = Integer.parseInt(request.getParameter("valor"));
response.getWriter().println("Valor Enviado: "+a+
                             "");
```

No fragmento de código apresentado, temos a recuperação de um parâmetro de nome **valor**, enviado através do protocolo HTTP, sendo recuperado no formato texto, a partir do método **getParameter** do objeto **request**, mas que teve a conversão para o formato inteiro com a chamada para **parseInt**. Em seguida, temos a escrita do conteúdo no canal de saída, através do objeto **response**, utilizando a sintaxe HTML.



Comentário

Algo que devemos ter em mente, com relação ao modelo Web, é a grande diferença do modelo desktop, no que se refere à gestão de estado de objetos. No modelo desktop, temos a armazenagem de objetos em um espaço de memória local, permitindo que os dados sejam mantidos durante toda a execução do sistema, mesmo que ocorra a troca de janelas, o que viabiliza a manutenção de estados, enquanto no modelo Web, a cada requisição e resposta, todos os objetos são recriados e o estado não pode ser mantido.

Uma solução primária para a manutenção de estados no HTTP é o uso de cookies, algo que pode ser explorado por meio do Java Script. Com o uso do modelo de cookies, temos a desvantagem de trabalhar com recursos locais da máquina do usuário, ficando sujeitos a bloqueios efetuados no navegador.

```
java

function setUser(login) {
    var d = new Date();
    d.setTime(d.getTime() + (24 * 3600 * 1000)); // 24 horas
    document.cookie = "username=" + login + ";expires=" +
                      d.toUTCString() + ";path=/";
}
function getUser() {
    var userStr = "username=";
    var ca = document.cookie.split(';');
    for(var i = 0; i < ca.length; i++) {
        var c = ca[i].trim();
        if (c.indexOf(userStr) == 0)
            return c.substring(userStr.length, c.length);
    }
    return "";
}
```

Nas funções de exemplo, com sintaxe Java Script, temos o uso do atributo **cookie**, no objeto **document**, para armazenar o usuário logado de forma local. O momento para expiração da informação é calculado em milissegundos, sendo definido um prazo total de 24 horas, a partir do qual teremos a invalidação do cookie.

Cookie

É um pequeno arquivo de computador ou pacote de dados enviado por um site de internet para o navegador do usuário. Cada vez que o usuário visita o site novamente, o navegador envia o cookie de volta para o servidor para notificar atividades prévias do usuário.



Dica

Com base em uma tecnologia servidora, a melhor solução para manter os estados será a utilização de sessões, que correspondem a objetos alocados ao servidor, fazendo referência a dada conexão. Enquanto o usuário se mantiver no site, todos os dados atribuídos serão mantidos, sendo eliminados na perda da conexão.

No ambiente Java, um objeto da classe **HttpSession**, que costuma se chamar **session**, permite a gerência de sessões HTTP, mas é sempre importante lembrar que ocorre consumo de memória no servidor, e a prática deve ficar restrita a finalidades específicas.

O servidor web Tomcat

Quando nos referimos ao **Tomcat**, estamos tratando de um projeto da Apache Software Foundation voltado para a definição de um servidor **Web**, ou **Web Server**, com código aberto e uso de tecnologias Java, como Java Server Pages (JSPs), Servlets, WebSockets e Java Expression Language.

O conjunto de tecnologias adotado pelo Tomcat oferece um ambiente consistente para responder às chamadas HTTP, além de prover suporte a **Servlets** e **JSPs** de forma nativa no **contêiner Web**. Por ser um produto de código aberto, acabou se tornando o padrão para hospedagem de sistemas Java para Web, oferecendo a possibilidade de executar de modo totalmente independente, ou atuando como módulo plugável em servidores de aplicativos Java, uma estratégia adotada tanto pelo **JBoss** quanto pelo **GlassFish**.

Contêiner

Ambiente de execução fechado voltado para tipos específicos de componentes.



Atenção

Um componente que implemente a interface Servlet é uma classe que permite ampliar as funcionalidades básicas de um servidor, ou seja, na prática é um aplicativo plugável, que deve ser executado em ambiente específico, como o contêiner Web oferecido pelo Tomcat. Por meio de um Servlet, temos o processamento no servidor, permitindo a geração dinâmica de conteúdo para a resposta HTTP.

Quanto às páginas **JSP**, elas permitem uma sintaxe em que a porção estática da resposta é definida com base em **HTML** ou **XML**, e trechos dinâmicos são intercalados através de fragmentos de código Java denominados **Scriptlets**, que são executados no servidor. Apenas a forma de escrita é modificada, pois as páginas JSP são

transformadas em Servlets pelo contêiner Web, quando ocorre o primeiro acesso, o que nos leva ao entendimento de que servidores como o Tomcat sempre utilizam Servlets para a geração de conteúdo dinâmico.

A complexidade da arquitetura do **Tomcat** é refletida na grande quantidade de arquivos e diretórios gerados na instalação padrão. Os principais diretórios do Web Server Tomcat servidor podem ser:

bin

Binários estruturais do servidor, agrupados em arquivos no formato jar, e scripts para inicialização ou término da execução.

conf

Arquivos de configuração, como server.xml, que guardam os parâmetros gerais do Tomcat, incluindo a porta utilizada para comunicação.

lib

Bibliotecas de inicialização do servidor, no formato jar, que também ficam disponíveis para todos os aplicativos do ambiente.

logs

Arquivos de log, extremamente úteis para a identificação dos erros que ocorreram durante a execução do servidor.

webapps

Diretório de base para a instalação dos aplicativos Java Web, ou seja, define a raiz do site. Cada aplicativo corresponderá a um subdiretório.

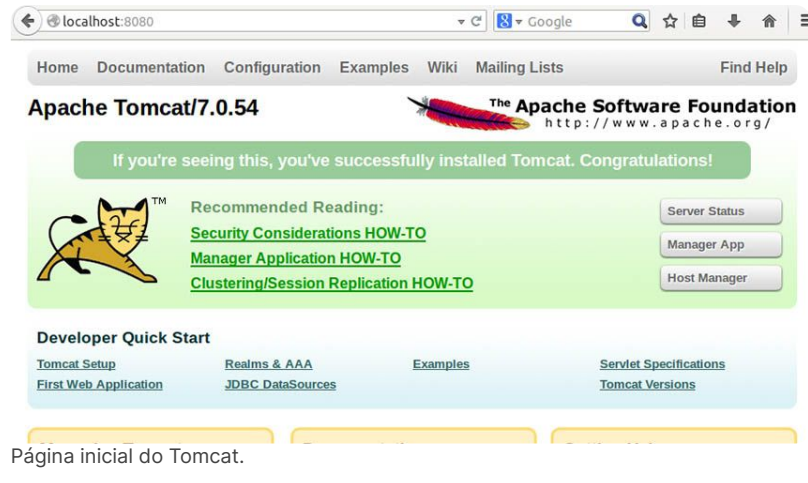
O **servidor Tomcat** pode ser utilizado como um serviço, executado a partir do ambiente do NetBeans, ou chamado diretamente pela linha de comando. Para executar na linha de comando, é necessário apontar a variável de ambiente **JAVA_HOME** para o local do **JDK** e executar o arquivo **startup.bat**, no Windows, ou **startup.sh**, para sistemas UNIX.

java

```
set JAVA_HOME="C:\\Java\\jdk\\jdk1.7.0_80"  
startup.bat
```

Com o servidor em execução, ele pode ser testado facilmente com a abertura de um navegador e a chamada para localhost:8080.

Com a abertura da página inicial do Tomcat, sabemos que a inicialização se completou corretamente e o servidor está disponível para a inclusão de aplicativos. O término da execução ocorrerá com a chamada para **shutdown.bat**, no Windows, ou **shutdown.sh**, em sistemas com base no UNIX. Veja a seguir:



No arquivo **server.xml**, do diretório **conf**, temos a possibilidade de alterar muitas das configurações do servidor, como o sistema de autenticação e a porta utilizada para a comunicação. A alteração da porta de conexão é feita no atributo com o nome **port**, presente nos elementos **Connector**, lembrando que o valor padrão é **8080**.

java

Outro arquivo de grande importância, com o nome **tomcat-users.xml**, está presente no diretório **conf**. Para a autenticação padrão, podemos definir os usuários, as senhas e os perfis do servidor com o uso desse arquivo.

plain-text

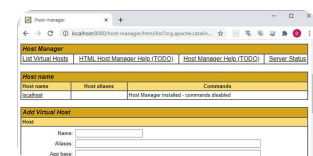
Na configuração inicial, temos apenas o usuário padrão do Tomcat, com utilização dos perfis, ou regras (**roles**), que liberam as funções administrativas apenas em modo texto. Podemos efetuar uma alteração muito interessante no arquivo **tomcat-users.xml**, acrescentando os perfis **manager-gui** e **admin-gui** ao usuário **tomcat**.

plain-text

Com a alteração efetuada, podemos acessar os **gestores gráficos**, clicando em alguns dos links presentes na página inicial do servidor, como **Manager App** e **Host Manager**.

Interface administrativa do TOMCAT

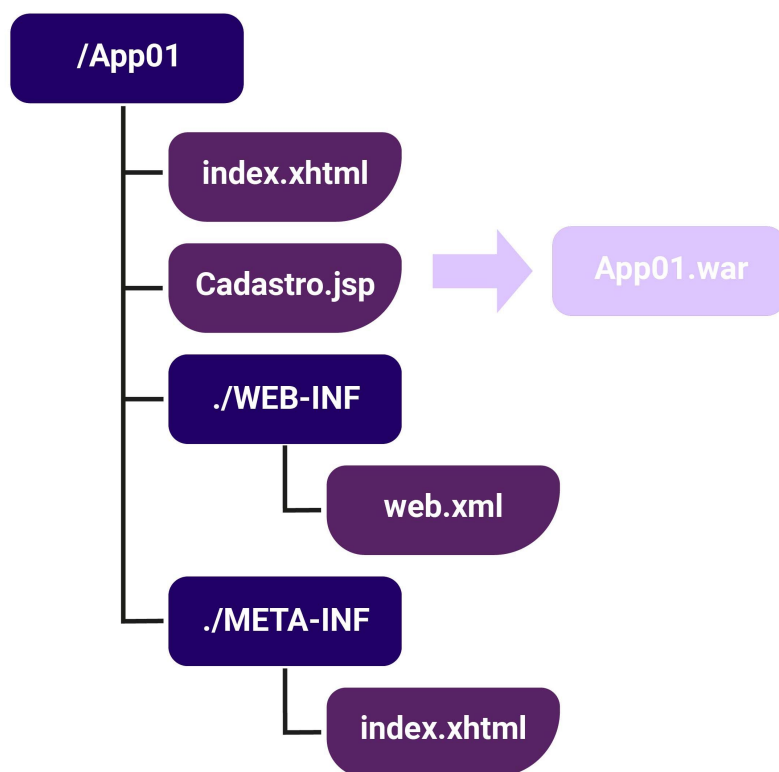
Por meio dos gestores gráficos, é possível criar servidores virtuais, gerenciar os aplicativos, implantar novos aplicativos, entre diversas outras funções administrativas, sem que haja a necessidade de alterar diretamente os arquivos XML de configuração.



Aplicativos web para o Tomcat

Estrutura do aplicativo web

Os aplicativos que criaremos deverão obedecer à estrutura exigida pelo Tomcat, que é composta por um diretório de base, ou raiz, com o nome do aplicativo, contendo um subdiretório de nome WEB-INF e outro com o nome META-INF. Vejamos na figura a seguir como se organiza a estrutura de um aplicativo Web Java:



Estrutura de aplicativo Web Java.

Vamos compreender melhor a seguir:

1

Diretório raiz

Temos páginas **JSP**, **HTML**, **XML**, e outros formatos interpretados, além de podermos acrescentar subdiretórios para a organização geral do conteúdo. É comum, por exemplo, definir diretórios para imagens, folhas de estilo e arquivos de código Java Script, permitindo que as bibliotecas e os temas utilizados sejam isolados em diretórios próprios, como no caso do **jQuery**, o que facilita a atualização de versões e a mudança do aspecto geral do sistema.

2

Diretório WEB-INF

Encontramos o arquivo de configuração com o nome **web.xml**, um subdiretório com o nome **classes** para os arquivos compilados do Java e um subdiretório **lib** com as bibliotecas nos formatos jar e zip. Quando criamos um **Servlet**, o arquivo compilado da classe Java é copiado para **classes**, obedecendo à estrutura de diretórios definida pelo pacote, e o reconhecimento pelo servidor pode ser configurado, com a utilização do mapeamento correto, no arquivo **web.xml**.

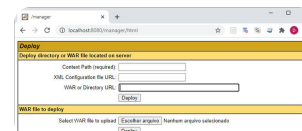
3Diretório META-INF

São definidas as configurações de ambiente, com recursos oferecidos para o aplicativo a partir do contêiner Web. Podemos ter arquivos como **context.xml**, com informações gerais de contexto, incluindo o pool de conexões com o banco de dados.

Vejamos alguns elementos fundamentais para a configuração do servidor.

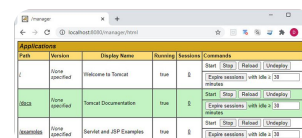
Área do manager app para implantação de aplicativos

Toda a estrutura criada pode ser compactada em um arquivo com extensão war, que significa **Web Archive**, podendo ser implantado no servidor, com o simples upload na divisão **WAR file to deploy** do aplicativo **Manager App**. De acordo com a configuração utilizada no servidor, quando temos a opção de **hot deployment** ativada, podemos copiar o arquivo war, ou toda a estrutura aberta, para um diretório específico, normalmente chamado de **webapps** ou **deploy**, conforme a distribuição, e o aplicativo será expandido, ficando disponível para os usuários.



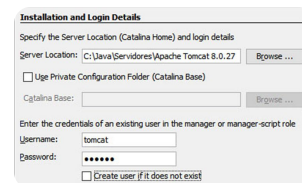
Área do manager app para acompanhamento de aplicativos

Com a configuração de hot deployment ativada, a desinstalação de um aplicativo ocorre por meio da simples remoção do diretório ou do arquivo war. Também é possível efetuar a desinstalação do aplicativo através da opção Undeploy de Manager App.



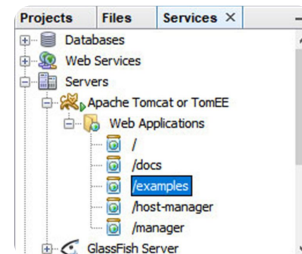
Acréscimo do servidor à interface do NETBEANS

A interface oferecida pelo **NetBeans** permite iniciar e parar o servidor, ou ainda remover os aplicativos implantados, por meio de uma interface gráfica simples, oferecida na aba **Services**, opção **Servers**. Caso o servidor não esteja configurado na plataforma, basta adicioná-lo, indicando diretório de instalação, usuário e senha.



Gerenciamento de servidores por meio do NETBEANS

Com o servidor configurado, basta clicar com o botão direito e escolher a opção **Start**, iniciando a execução e permitindo acessar e remover os aplicativos implantados, o que também é feito a partir do clique com o botão direito. A implantação do aplicativo, por sua vez, ocorre a partir da execução de um projeto do tipo Web.



Configuração do Web Server Tomcat

Assista ao vídeo a seguir para saber mais sobre o ambiente Java e a instalação e funcionalidades do Web Server Tomcat.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Verificando o aprendizado

Questão 1

Quando criamos um aplicativo Web na plataforma Java, o Tomcat é muito utilizado como Web Server, e a preferência é justificada por algumas de suas características estruturais e funcionais. Qual das opções apresentadas NÃO pode ser considerada uma característica do Tomcat?

A

Suporte a diversas tecnologias Java, incluindo Servlets e JSPs.

B

Uso de código fechado.

C

Apresenta uma configuração muito flexível, a partir de arquivos XML.

D

Pode trabalhar como módulo plugável de outros servidores, como o GlassFish.

E

É o padrão para hospedagem de sistemas Java para web.



A alternativa B está correta.

O servidor Apache Tomcat é uma implementação de código aberto das tecnologias Java Servlet, Java Server Pages, Java Expression Language e Java WebSocket.

Questão 2

Uma característica do protocolo HTTP é a de que não ocorre manutenção de estados entre chamadas sucessivas, mas trabalhando em um ambiente servidor, temos um recurso específico para manter valores entre chamadas sucessivas, sem utilização de persistência no cliente. Qual o nome do recurso descrito?

A

Requisição

B

Resposta

C

Cookie

D

Sessão

E

Retorno



A alternativa D está correta.

Quando trabalhamos com uma tecnologia servidora, podemos utilizar um recurso de sessão (***session***) para guardar valores, referentes à conexão, na memória do servidor, o que é diferente do uso de cookies, quando temos apenas uma persistência de dados simples ocorrendo no cliente.

Visão geral

Servidores de aplicativos

Para começar, assista ao vídeo a seguir para aprender mais sobre servidores de aplicativos.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Quando nos referimos a servidores de aplicativos, ou **Application Servers**, estamos considerando plataformas capazes de suportar o perfil de processamento necessário para projetos de grande porte, com alto nível de conectividade, processamento paralelo e distribuído, *pools* de recursos compartilhados, serviços de autenticação e autorização, entre diversos outros elementos comuns para um ambiente empresarial complexo. Entre os componentes necessários, é comum encontrar os **Web Servers**, que nesse caso atuam como módulos internos dos **Application Servers**.



Comentário

O objetivo primário de um servidor de aplicativos é prover um ambiente que trate das necessidades comuns do ambiente de execução, como segurança, balanceamento de carga, alta disponibilidade, gerenciamento de exceções e controle transacional, entre outras. Esses servidores oferecem uma arquitetura e um conjunto de contêineres, permitindo ao programador se concentrar apenas nas regras de negócio e deixando para o servidor a responsabilidade sobre a gerência de recursos e funcionalidades comuns.

Servidores de aplicativos podem trabalhar de forma conjunta, com base em protocolos voltados para o processamento distribuído, o que traz mais dificuldades na gerência de **exceções**, já que podem ocorrer de forma local ou remota. Com a possibilidade da ocorrência de exceções em vários locais distintos, dentro de um mesmo processo, o controle transacional também precisa englobar esses locais, levando à necessidade de bibliotecas para a gerência de **transações distribuídas**.



Atenção

Uma transação é um processo atômico e isolado, que deve ser executado de forma consistente e com resultados duráveis.

Uma necessidade básica do ambiente corporativo é a gerência da **segurança**, exigindo ferramentas robustas para a **autenticação** de usuários e **autorização** para o uso de recursos e funcionalidades. Além de gerenciar o acesso, nosso servidor deve ser capaz de oferecer protocolos seguros, como **TLS** (Transport Layer Security).

Como os servidores de aplicativos devem lidar com diversos fluxos de dados que fazem parte do ambiente corporativo, incluindo **bases de dados** relacionais e **mensagerias**, um requisito primário é a presença de um grande conjunto de bibliotecas de *middleware*, garantindo a interoperabilidade com diversos outros sistemas. É com base nas opções de *middleware* que são definidos alguns componentes, como *pools* de conexões com o banco de dados e canais de comunicação com mensagerias, entre outros.



Comentário

Devido à grande quantidade de componentes e recursos compartilhados, é natural que haja a necessidade de individualização, o que deve ser feito com o uso de nomes únicos, gerenciados por serviços de nomes e diretórios. Para servidores criados na plataforma Java, o JNDI (Java Naming and Directory Interface) gerencia os serviços para registro e localização de recursos, funcionando como um canal de identificação comum entre os aplicativos, tanto internamente quanto para acessos externos, além de centralizar toda a integração com serviços de nomes e diretórios de outras plataformas.

Temos diversas opções de servidores de aplicativos criados no ambiente Java, como JBoss, **GlassFish**, WebSphere, Oracle Application Server e WebLogic, sempre com base no **JEE** (Java Enterprise Edition), uma arquitetura de referência para a implementação de ambientes de execução corporativos, com suporte a objetos distribuídos.

Objetos distribuídos e o Glassfish

Tecnologias de objetos distribuídos

Vamos entender mais sobre tecnologias de objetos distribuídos? Assista ao vídeo a seguir.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Um ponto central, na grande maioria dos **Application Servers**, é a presença de sistemas de **objetos distribuídos**, como **CORBA** (**Common Object Request Broker Architecture**), ou Microsoft **DCOM** (**Distributed Component Object Model**). No caso do Java, temos os componentes do tipo **EJB** (**Enterprise Java Bean**).

Quando nos referimos a processamento distribuído, estamos considerando serviços que são oferecidos de forma remota, ou distribuída, usando protocolos próprios, como **RPC** (Remote ProcedureCall) e **RMI** (Remote Method Invocation). Qualquer que seja a tecnologia para processamento distribuído, alguns elementos são comuns para a definição da maioria das arquiteturas:

- Protocolo de comunicação
- Serviço de registro e localização
- Descritor de serviços

Um exemplo comum de processamento distribuído, no mercado de desenvolvimento, são os **Web Services**, como o tipo **SOAP** (**Simple Object Access Protocol**), que utiliza um protocolo de mesmo nome, **WSDL** (**Web Service Description Language**) para descrever os serviços e **UDDI** (**Universal Description, Discovery and Integration**) nas tarefas de registro e localização.

Algo muito interessante sobre os elementos citados é que todos trabalham com **XML**, garantindo a interoperabilidade em todos os níveis da arquitetura.

Quando consideramos os objetos distribuídos, temos o mesmo tipo de fornecimento de serviços, de forma distribuída, mas agora com a execução efetuada a partir de *pools* de objetos nos servidores. Por exemplo, no CORBA temos a utilização do protocolo de rede **IIOP** (**Internet Inter-ORB Protocol**), descritor **IDL** (**Interface Definition Language**), além de registro e localização via **COS Naming**, lembrando que COS significa CORBA **Object Services**.



Atenção

O registro dos EJBs, no ambiente Java, utiliza JNDI, e a comunicação utiliza o protocolo misto RMI-IIOP. Quanto ao descritor de serviços, utilizamos interfaces Java, mas pode ser gerado o IDL a partir das interfaces.

A utilização dos serviços distribuídos segue um fluxo comum:

- Localizar o serviço ou objeto distribuído a partir do serviço de nomes.
- Recuperar o descritor a partir da localização encontrada.
- Gerar o cliente de comunicação a partir do descritor.
- Transmitir as solicitações, através do cliente, com base no protocolo.

Com a utilização da ferramenta correta, o cliente é gerado de forma automática a partir do descritor, permitindo que nossas chamadas ao cliente se assemelhem a solicitações locais, enquanto toda a comunicação remota ocorre de forma transparente.

Durante a execução dos objetos, no *pool* do servidor, o contêiner irá fornecer todos os recursos necessários em termos de autenticação, autorização, acesso a **middleware**, controle transacional, entre outras funcionalidades. Com base em um contêiner, temos a execução controlada e robusta dos processos de negócio do nosso sistema.

Confira a seguir os **componentes arquiteturais** para alguns dos serviços distribuídos mais comuns.

TECNOLOGIA	REGISTRO E LOCALIZAÇÃO	DESCRITOR	PROTOCOLO
CORBA	COS Naming	IDL	IIOP
SOAP	UDDI	WSDL	SOAP
RMI	JNDI	Interface Java	RMI
EJB	JNDI	Interface Java	RMI-IIOP

Tabela: Componentes arquiteturais para serviços distribuídos. Denis Gonçalves Cople

Application server Glassfish

Ferramentas do servidor de aplicações Glassfish

Assista ao vídeo a seguir para saber mais sobre a instalação e as principais ferramentas do console administrativo do servidor de aplicações Glassfish.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Enquanto o Tomcat suporta, de forma nativa, apenas Servlets e JSPs, atuando como um **Web Server**, o **GlassFish** vai além, oferecendo suporte às tecnologias Java de objetos distribuídos, no caso os **EJBs**, sendo classificado como **Application Server**.

É importante mencionar que o Tomcat é utilizado pelo GlassFish como módulo interno, delegando para o **Web Server** as tarefas de comunicação no protocolo HTTP e tratamento de Servlets e JSPs, enquanto os outros componentes da arquitetura do GlassFish tratam das diversas tecnologias do **JEE** (Java Enterprise Edition).



Arquitetura enterprise

Com base no GlassFish, somos capazes de criar sistemas mais complexos, com uso de EJBs e transações distribuídas, além de obtermos ferramentas para gerenciamento de componentes corporativos, como mensagerias. O servidor também disponibiliza um ambiente de testes simplificado para **Web Services** do tipo SOAP.



Atenção

Seguindo a arquitetura JEE, o ponto central do servidor é o contêiner EJB, que gerencia o ciclo de vida dos objetos distribuídos, além de intermediar a utilização de recursos do ambiente. A disponibilização de recursos para os EJBs é configurada através de arquivos XML e anotações de código, sempre por meio de identificadores no JNDI.

Como todas as bibliotecas de *middleware* são gerenciadas pelo servidor, o processo é o mesmo para a grande maioria dos recursos. De modo geral, temos a criação do recurso, como um *pool* de conexões, por exemplo, e o registro no **JNDI**, em um primeiro instante, e a localização e utilização pelo contêiner, a partir das configurações definidas para os EJBs, no momento da execução do aplicativo.

Entre as bibliotecas e os elementos gerais de *middleware* do GlassFish, podemos destacar a presença do **JDBC** (Java Database Connectivity), intermediando o acesso aos bancos de dados, e do **JMS** (Java Message Service), gerenciando o uso de mensagerias a partir de canais e filas, além de bibliotecas para o controle de transações locais e distribuídas, como **JTA** (Java Transaction API) e **JTS** (Java Transaction Service).

Além dos componentes do GlassFish, e do Tomcat como módulo, a distribuição padrão inclui o serviço de mensageria **GlassFish Message Queue**, o banco de dados **Derby**, que também é conhecido como **Java DB**, e uma ferramenta de inicialização pelo console.

Vejamos a seguir os níveis de diretórios do **Application Server GlassFish**:

1

bin

Contém apenas a ferramenta **asadmin** para inicialização pelo console e representa a divisão descrita.

2

glassfish

Diretório do servidor de aplicativos, com os componentes estruturais, arquivos de configuração e domínios estabelecidos.

3

javadb

Corresponde a uma distribuição do banco de dados Derby, contendo os subdiretórios **bin** e **lib**, com scripts de gerenciamento e bibliotecas.

4

mq

Contém os componentes da mensageria GlassFish MQ, além de diversos exemplos de utilização, disponibilizados no subdiretório **examples**.

Ao trabalhar com o **servidor GlassFish**, precisamos definir **domínios**, que são ambientes completos e independentes para a instalação e execução de aplicativos. Fazendo uma comparação com sistemas de gerenciamento de bancos de dados, como o Oracle, temos o equivalente a **instâncias** de execução.

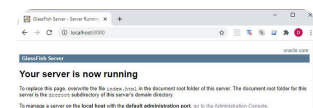
Instalação do Glassfish

Um domínio padrão, com o nome **domain1**, é criado, e podemos iniciar o servidor a partir da linha de comando com o uso de **asadmin**, tendo como parâmetros a opção **start-domain** e o nome do domínio.

```
asadmin start-domain domain1
```

Tela inicial do Glassfish

Com o servidor iniciado, podemos acessar o domínio por meio da chamada para **localhost** na porta configurada, com o valor padrão **8080**, através de um navegador, enquanto o console administrativo utiliza a porta **4848**. Para encerrar a execução, basta invocar o mesmo programa (**asadmin**), com a utilização do parâmetro **stop-domain**, sendo ainda possível consultar os domínios cadastrados por meio da opção **list-domains**.

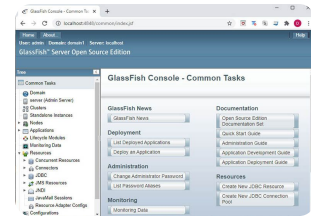


Interface administrativa do Glasfish

Observando a árvore de diretórios do servidor, temos subdiretórios para bibliotecas compartilhadas, módulos estruturais e programas administrativos (**bin**), entre outros. O mais importante para nós será o subdiretório **domains**, onde são definidas as estruturas de domínio do servidor. Entre os subdiretórios do domínio, temos um para a definição de configurações (**config**), podendo ser observado o arquivo **domain.xml**, com a definição de características gerais, como as portas de conexão utilizadas.

Basicamente, apenas as portas de conexão precisam ser modificadas de forma manual para o domínio, ao nível do elemento **network-listeners**, de **domain.xml**, sendo todas as demais configurações efetuadas através do console administrativo, na porta **4848**.

Por meio da interface administrativa, é possível executar diversas tarefas, como implantar os aplicativos, criar os pools de conexão com o banco de dados, gerenciar filas e canais da mensageria, definir identificadores JNDI, entre outras.



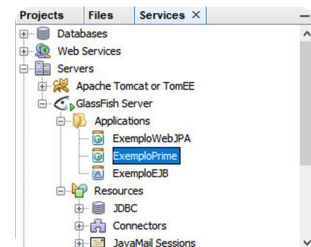
Gerenciamento de Clusters no Glassfish

Por meio da árvore de navegação, iniciada em **Common Tasks**, temos acesso rápido a todas as tarefas administrativas, como o gerenciamento de **Clusters**, por exemplo.



Gerenciamento do Glassfish por meio do Netbeans

Assim como no Tomcat, podemos acessar e configurar elementos básicos do GlassFish a partir do **NetBeans** por meio da aba **Services**.



Aplicativo corporativo

Um aplicativo corporativo comporta elementos como **EJBs**, **Servlets** e **JSPs**. De forma geral, os objetos de negócio e as classes de acesso ao banco de dados são compactados em um arquivo **jar**, enquanto a interface Web é organizada em um arquivo **war**, sendo os dois arquivos compactados em um terceiro, com extensão **ear** (Enterprise Archive).

A estrutura padrão para implantação é obedecida pelo NetBeans, quando criamos um projeto do tipo **Enterprise Application**.

Na prática, são criados um projeto principal, com extensão **ear**, e dois projetos internos, com as extensões **jar** e **war**. Para implantar e testar o aplicativo, devemos utilizar sempre o projeto principal, representado por um triângulo.

Uma estratégia simples de programação, dentro da arquitetura imposta pelo servidor, envolve a programação de classes **DAO** (Data Access Object) para acesso ao banco de dados, e **EJBs**, com as regras de negócio, no projeto **jar**. Como o projeto trata da camada de negócios do sistema, não devem ser definidos elementos de interface neste nível.



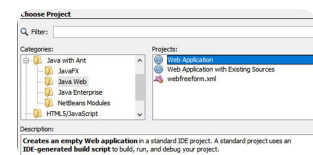
Atenção

Com a camada de negócios completa, ao nível do jar, podemos implementar a interface Web no projeto war, com a criação de elementos como Servlets e JSPs. Os elementos de interface não podem conter inteligência de negócios, funcionando como um simples canal de comunicação com o usuário e direcionador das solicitações efetuadas para a camada de negócios.

Projetos mais simples, que não envolvem os **EJBs** e demais elementos corporativos, não precisam de um modelo tão robusto e podem ser definidos em termos de um simples aplicativo para Web, com extensão **war**. Nesse caso, os objetos de negócios e as classes de acesso ao banco de dados ficam no mesmo projeto, junto aos Servlets e JSPs.

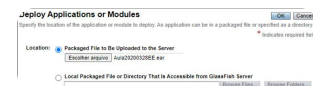
Criação de aplicativo web no Netbens

Para a criação de um aplicativo Web simples, que pode ser executado tanto no Tomcat quanto no GlassFish, utilizamos um projeto do tipo **Web Application**. Ao executar um projeto Web ou corporativo, ele é implantado no servidor e exibido no navegador após a implantação. Se o servidor não estiver ativo, o NetBeans irá iniciá-lo automaticamente, antes de implantar o aplicativo.



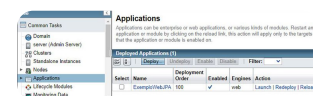
Implantação de aplicativo no console do Glassfish

Para também implantar o aplicativo em um servidor externo, pode-se usar o console de administração do GlassFish, com o uso da opção **Deploy an Application**. Tudo o que precisamos fazer, na tela, é escolher o arquivo, com extensão **ear** ou **war**, e com o clique no botão **OK** será efetuada a implantação.



Gerenciamento de aplicativos no console do Glassfish

Para verificar os aplicativos instalados, basta acessar a opção **Applications** na árvore de navegação, onde são listados todos os nossos aplicativos, sendo possível ainda efetuar a remoção deles. Em termos práticos, devido à funcionalidade do **autodeploy**, temos a implantação do aplicativo com o simples **upload** de um arquivo, e a remoção ocorre com a exclusão do arquivo no servidor.



Verificando o aprendizado

Questão 1

Em um ambiente corporativo, os objetos distribuídos são muito importantes para garantir a robustez do sistema, com o uso colaborativo de funcionalidades processadas de forma distribuída. Como estamos lidando com elementos em servidores distintos, é necessário um protocolo adequado, como no caso do CORBA, que utiliza o protocolo:

A

IIOP

B

SOAP

C

RMI

D

RMI-IIOP

E

RPC



A alternativa A está correta.

A tecnologia CORBA foi uma das primeiras opções de objetos distribuídos aceita comercialmente. No ambiente CORBA, temos o descritor de serviços do tipo IDL, o registro e a localização pelo COS Naming, e o protocolo IIOP.

Questão 2

Quando utilizamos um servidor GlassFish, obtemos acesso a um grande conjunto de tecnologias Java, disponibilizado por meio de contêineres próprios. Dentre as diversas tecnologias presentes, qual delas é voltada para a comunicação com mensagerias?

A

JDBC

B

JMS

C

JTA

D

JTS

E

JEE



A alternativa B está correta.

A tecnologia JMS (**Java Message Service**) constitui o **middleware** do ambiente Java para acesso a recursos de mensagerias. É um tipo de **middleware** que recebe a denominação MOM, ou **Message Oriented Middleware**.

Aplicativos web no Netbeans

Neste vídeo, vamos aprender sobre os aplicativos web no Netbeans.

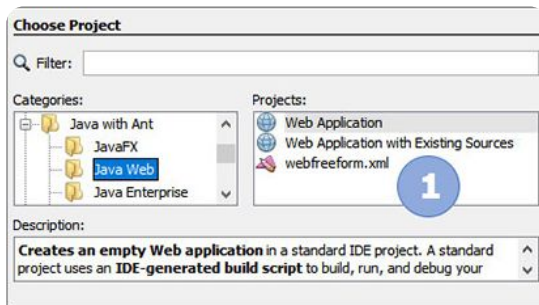


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Em nosso contexto de estudos, um aplicativo corporativo não será necessário, já que estamos analisando tecnologias voltadas para a camada Web. Precisaremos apenas de um aplicativo Web, com extensão **war**, e o uso de um servidor, como **Tomcat** ou **GlassFish**.

Vamos criar um **projeto de testes**, conforme os passos apresentados a seguir:



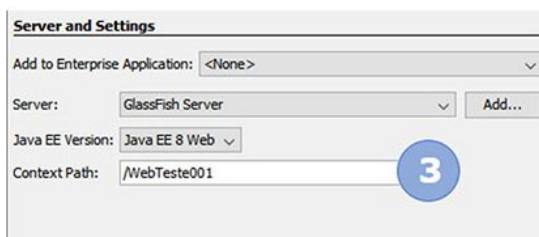
Passo 1

Escolha o projeto do tipo **Web Application**, na categoria **Java Web**.



Passo 2

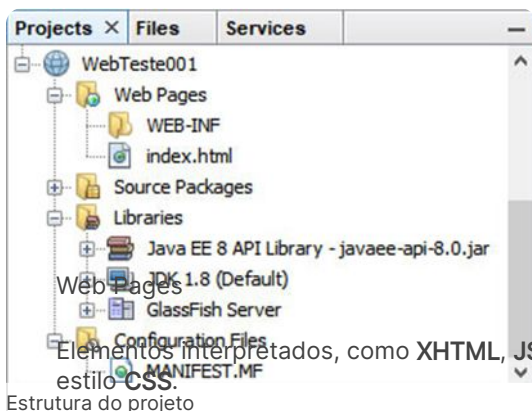
Defina o nome do projeto (**WebTeste001**).



Passo 3

Escolha o servidor (**GlassFish**) e a versão do JEE (**Java EE 8 Web**).

Ao término da execução desses passos, o novo projeto será apresentado na aba **Projects** do NetBeans. Pode ser necessário acrescentar manualmente a biblioteca **Java EE 8 API**, caso não esteja instalada.



Observe que o arquivo **web.xml** não é apresentado na estrutura do projeto, o que se deve ao fato de termos utilizado o **JEE versão 8**, onde as configurações são efetuadas, em sua grande maioria, através de anotações no código Java.

A seguir, apresentaremos algumas informações adicionais:

Source Packages

Elementos compilados, definidos em termos de pacotes e classes Java, incluindo nossos **Servlets**.

Libraries

Bibliotecas Java requeridas pelo aplicativo Web.

Configuration Files

Arquivos de configuração, como **MANIFEST.MF** e **web.xml**.

Agora podemos começar a criar os elementos constituintes de nosso sistema, incluindo **Servlets** e **JSPs**, e executar o projeto. Na execução, teremos a geração do arquivo **war**, o servidor será iniciado, caso ainda não esteja ativo, e o arquivo compilado será copiado para o diretório de **autodeploy**, ocorrendo a implantação do aplicativo e abertura do navegador no endereço correspondente ao **index.html**.

Tecnologias Servlets e JSPS

Tecnologia de Servlets

Vamos aprender mais sobre tecnologia de Servlets? Assista ao vídeo a seguir.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Podemos dizer que **Servlet** é um componente para a criação de serviços remotos, que estendem a funcionalidade de algum servidor específico. A tecnologia de Servlets foi criada como uma solução genérica para a definição de aplicativos hospedados em servidores, quaisquer que fossem os protocolos utilizados por eles, mas a especialização para o **HTTP** foi a que se tornou mais popular

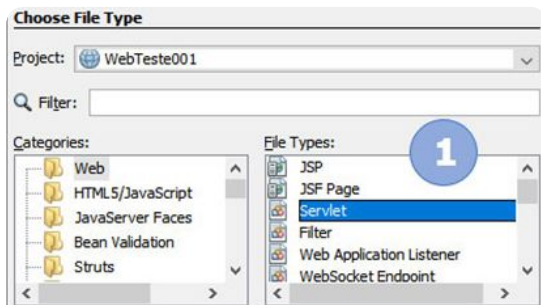


Comentário

A classe `HttpServlet`, filha de `GenericServlet`, integra-se ao ambiente de execução do servidor através do contêiner Web, tirando proveito das tecnologias Java presentes ali, como pools de conexão, controle transacional e ferramentas de segurança. Quanto à comunicação com o usuário, a classe `HttpServlet` apresenta métodos para responder às chamadas dos tipos GET e POST no protocolo HTTP.

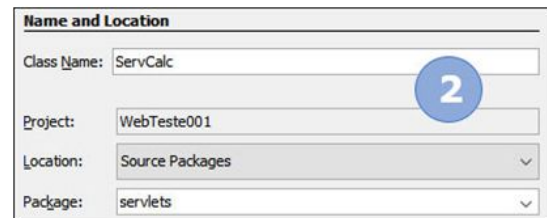
Tudo que precisamos fazer é criar um descendente da classe **HttpServlet**, herdando toda a integração já existente com o ambiente, e alterar os métodos **doGet** e **doPost** para, através do polimorfismo, personalizar as respostas em nosso aplicativo às chamadas dos tipos **GET** e **POST**, respectivamente.

Para definir um **Servlet no NetBeans**, adicione um **novo arquivo** e siga os passos apresentados:



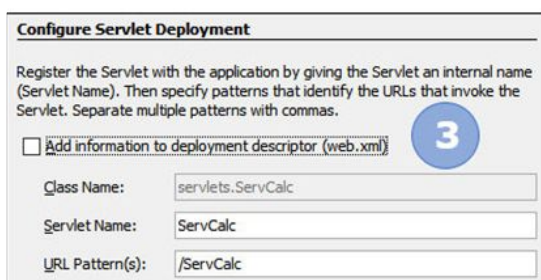
Passo 1

Escolha o arquivo do tipo **Servlet** na categoria **Web**.



Passo 2

Defina o nome do Servlet (**ServCalc**) e do pacote (**servlets**).



Pssso 3

Finalize sem adicionar as informações ao arquivo **web.xml**.

Teremos o código de **ServCalc** no **editor de código** do NetBeans e devemos observar alguns detalhes, como o **código oculto** (editor-fold), que trata de um trecho de código escondido por meio de comentários no início e fim do bloco, com a visualização alternada através do clique sobre o sinal de **adição**, próximo à margem esquerda.

O código oculto, para **ServCalc**, engloba os métodos **doGet** e **doPost**, responsáveis pela recepção de chamadas **HTTP** dos tipos **GET** e **POST**, respectivamente. No código gerado, ambos os métodos redirecionam para **processRequest**, fazendo com que GET e POST sejam tratados da mesma forma, algo que nem sempre é interessante, pois temos situações, como no processo de login, em que não devemos aceitar chamadas GET.

O próximo passo será a alteração do método **processRequest**, para que corresponda ao processamento necessário em nosso aplicativo.

```
java
protected void processRequest(HttpServletRequest request,
                             HttpServletResponse response)
    throws ServletException, IOException {
    response.setContentType("text/html;charset=UTF-8");
    try (PrintWriter out = response.getWriter()) {
        int a = new Integer(request.getParameter("a"));
        int b = new Integer(request.getParameter("b"));
        out.println("");
        out.println("A soma de " + a + " e " + b + " será " +
                    (a + b));
        out.println("");
    }
}
```

A assinatura do método **processRequest**, assim como ocorre em **doGet** e **doPost**, traz dois tipos de parâmetros:

Tipo **HttpServletRequest**

Com o nome **request**, para receber a requisição do usuário.



Tipo **HttpServletResponse**

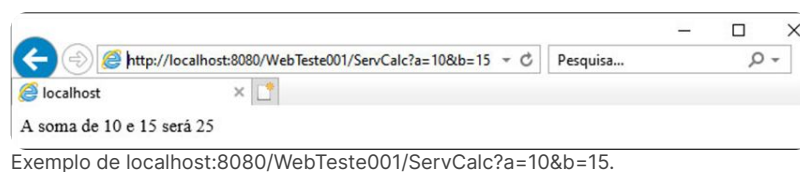
Com o nome **response**, que será responsável pela resposta HTTP.

Iniciamos a implementação de **processRequest** com a definição do tipo de saída, que no caso é **HTML**, com codificação **UTF-8**. Em seguida, iniciamos um bloco protegido, do tipo **try with resources**, em que o objeto de nome **out** recebe o canal de saída de **response**.

Em meio ao bloco protegido, temos a recuperação dos parâmetros **a** e **b**, por meio do método **getParameter**, com a conversão para as variáveis inteiras de mesmo nome, o que é feito com base na classe **Integer**. Como o protocolo **HTTP** trabalha com valores do tipo texto, qualquer outro formato exigirá conversões ao nível do Java.

Após a **recuperação** e a **conversão dos parâmetros da requisição**, efetuamos a montagem da resposta através do canal de saída, com base em chamadas ao método **println**, como se estivéssemos enviando a informação para o console. Nossa resposta será uma página **HTML** simples, com a presença da frase com a soma dos valores fornecidos.

Para testar nosso Servlet, devemos executar o projeto e efetuar a chamada correta a partir do navegador, como no exemplo seguinte:



Devemos compreender como ocorre o direcionamento de fluxo, no protocolo HTTP, para a classe **ServCalc**. No formato original de mapeamento, deveríamos acrescentar uma entrada no arquivo **web.xml**, mas como estamos lidando com uma versão mais recente do contêiner, uma anotação **WebServlet**, aplicada à classe **ServCalc**, definirá o mapeamento necessário.

```
java
```

```
@WebServlet(name = "ServCalc", urlPatterns = {"/ServCalc"}) public class ServCalc extends  
HttpServlet {
```

A anotação **WebServlet** requer o **nome** do Servlet e o padrão de escrita da **URL** para o redirecionamento. Embora, tradicionalmente, sejam utilizados textos correspondentes ao nome da classe em si, essa não é uma regra, permitindo até mesmo a utilização de endereços dinâmicos na URL, com base em coringas, como no caso de **"*.jsf"**.

Para que não seja necessário escrever o endereço completo a cada chamada, podemos alterar o arquivo **index.html**.

```
plain-text
```

Finalizando a modificação do **index** e executando novamente o aplicativo, teremos um formulário para a entrada dos valores e o botão de envio para o Servlet.

Tecnologia de JSPS

Demonstração dos artefatos do JSP

Assista ao vídeo a seguir para saber mais sobre a criação de página JSP.

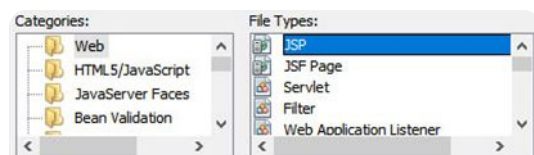


Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

A criação de **Servlets** seria suficiente para prover as necessidades de todo e qualquer aplicativo Java Web, porém a construção de páginas diretamente no código pode se tornar desconfortável para os profissionais de design. O problema foi solucionado com um novo modelo de programação, em que os códigos Java são escritos dentro do conteúdo **HTML** ou **XML** por meio da tecnologia **JSP** (Java Server Pages).

O processo para a criação de uma página **JSP** envolve apenas a adição de um arquivo, do tipo **JSP** na categoria **Web**. É necessário definir o nome do arquivo, **sem** acrescentar a extensão, e o novo componente ficará disponível na divisão **Web Pages** do projeto, ou em um subdiretório especificado nela.



Divisão Web Pages

Vamos criar uma **página JSP simples**, utilizando o nome **ListaCores**. Ela não irá fornecer nenhuma funcionalidade relevante para o sistema, mas permitirá que analisemos os elementos estruturais principais das páginas JSP.

plain-text

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```
    <%  
    String[] cores = {"vermelho", "verde", "azul"};  
    for(String x: cores) {  
        out.println("• "+x+"  
        ");  
    }  
    %>
```

Vamos agora compreender melhor Diretivas, HTML e Scriptlet a seguir:

Diretivas

A primeira linha deste código é uma **diretiva**, no caso indicando o tipo de conteúdo e a página de acentuação que será adotada. Diretivas também são utilizadas para importar bibliotecas e definir a página de erro, entre diversas outras opções.

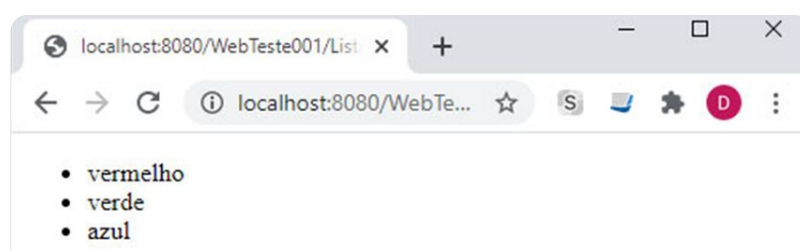
HTML

Nas **linhas seguintes**, temos código **HTML** padrão, que irá compor a saída sem requerer interpretação no servidor. O que temos aqui é a definição da estrutura básica da página HTML e a inclusão de uma lista, com o uso do marcador **UL**.

Scriptlet

Internamente ao marcador de lista, temos um bloco iniciado com os símbolos de menor e percentual e terminado com os símbolos de percentual e maior, definindo um trecho que é executado no servidor, com utilização de sintaxe Java, e que é conhecido como **Scriptlet**. No trecho de código do exemplo, temos a definição de um vetor de texto com nomes de cores, as quais serão impressas no conteúdo da página através do objeto **out**, implícito nas páginas **JSP**, como itens de lista.

Para testar nossa página JSP, vamos executar o projeto e efetuar a chamada correta a partir do navegador, como no exemplo seguinte. Estando tudo correto, teremos uma **página de resposta**.



Os objetos **request** e **response** também são implícitos para as páginas **JSP**, podendo ser utilizados da mesma forma que nos métodos `doGet` e `doPost` dos **Servlets**. Inclusive, devemos nos lembrar de que a página **JSP** é convertida em **Servlet** no primeiro acesso, logo o que muda é a forma de programar, e não a funcionalidade original.

Controle de sessões

As **sessões** HTTP são de grande utilidade no ambiente Web, provendo uma forma de manutenção de **estados** na troca de páginas, pois ao contrário dos sistemas desktop, com valores mantidos na alternância de janelas, a cada página temos um novo conjunto de variáveis, desconsiderando-se todas as existentes antes da requisição ser efetuada.

Podemos controlar sessões de forma muito simples, com o uso da classe **HttpSession**, e um exemplo típico de utilização é no controle de login. Normalmente, temos um **Servlet** para a verificação do login, e a sessão deve ser obtida a partir do objeto de requisição, com a invocação do método **getSession**.



Atenção

Nas páginas **JSP**, o controle de sessões é feito com o uso do objeto implícito `session`, da classe `HttpSession`.

Para demonstrar o processo de login, vamos criar uma página **JSP**, que receberá o nome "**Segura.jsp**" e representará um recurso com acesso autenticado.

```
plain-text
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%
    if(session.getAttribute("usuario")==null)
        response.sendRedirect("Login.jsp");
    else {
%>
```

Esta é uma página protegida!

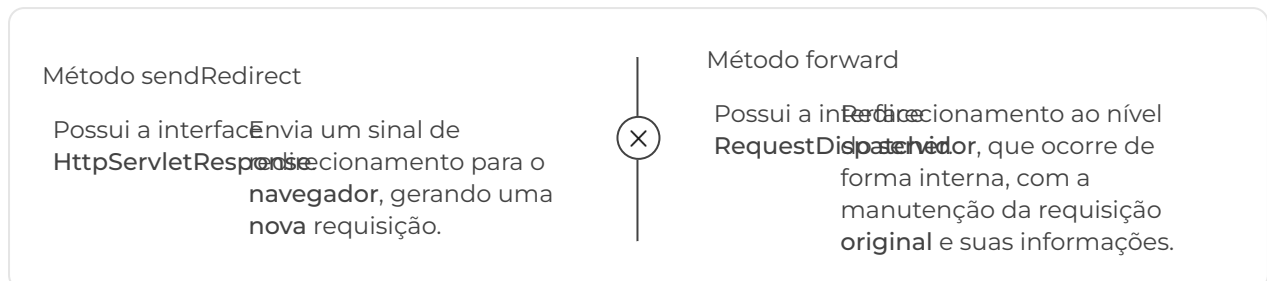
O usuário `<%=session.getAttribute("usuario")%>`
está logado.

```
<% } %>
```

Enquanto os **parâmetros** da requisição HTTP assumem apenas valores do tipo texto, os **atributos** da sessão permitem guardar qualquer tipo de objeto. Embora seja possível a utilização de estruturas mais complexas, iremos utilizar apenas um elemento de texto.

Na primeira parte do arquivo **JSP**, temos o teste para a existência do atributo "usuário" na sessão, e se ele não existir, isto significa que não há um usuário autenticado, devendo ocorrer o redirecionamento para a página de **login** através de **sendRedirect**.

A seguir, temos os dois métodos de **redirecionamento** do Java para Web.



Ainda observando o código da página JSP, podemos notar que a instrução **else** é aberta antes do início do código HTML e fechada apenas no final, logo após a tag de finalização, significando que a página de resposta será processada apenas se o atributo usuário estiver presente na sessão, ou seja, se existir alguém autenticado no sistema. Quando ocorre a montagem da página, temos um retorno bastante simples, com a exibição do login corrente, obtido através de **getAttribute**, e um botão para efetuar o logout, ou desconexão, por meio de uma chamada para **ServletLogin**.

Agora precisamos criar a **página de login**, que receberá o nome "**Login.jsp**", e o Servlet responsável pelo controle das ações referentes aos processos de conexão e desconexão, que será chamado de **ServletLogin**.

```
plain-text
<%@page contentType="text/html" pageEncoding="UTF-8"%>

Acesso ao Sistema

Login:
Senha:

<%
    if(request.getAttribute("erro")!=null) {
%>



---


Ocorreu um erro: <%=request.getAttribute("erro")%>
<%
    }
%>
```

Na primeira parte de **Login.jsp**, temos um formulário **HTML** bastante simples, contendo as informações que deverão ser enviadas para **ServletLogin** no processo de verificação, enquanto na segunda parte

apresentamos mensagens de erro de autenticação. Note que a segunda parte será apresentada apenas se o **atributo** de **erro** estiver presente na chamada ao JSP, e que, por se tratar de um atributo, e não de um parâmetro, não permite o envio a partir do protocolo HTTP, mas apenas por meio do código do Servlet.

```
java

@WebServlet(name = "ServletLogin",
            urlPatterns = {"/ServletLogin"})
public class ServletLogin extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {
        String acao = request.getParameter("acao");
        if(acao==null)
            throw new ServletException("Parâmetro Requerido");
        HttpSession session = request.getSession();
        switch(acao){
            case "conectar":
                if(request.getParameter("login").equals("admin")&&
                   request.getParameter("senha").equals("123")){
                    session.setAttribute("usuario", "Administrador");
                    response.sendRedirect("Segura.jsp");
                } else {
                    request.setAttribute("erro","Dados inválidos.");
                    RequestDispatcher rd =
                        request.getRequestDispatcher("Login.jsp");
                    rd.forward(request,response);
                }
                break;
            case "desconectar":
                session.invalidate();
                response.sendRedirect("index.html");
                break;
            default:
                throw new ServletException("Parâmetro incorreto");
        }
    }
}
```

Aqui estamos tratando de um processo de login muito simples, mas que, por se tratar de uma autenticação de usuário, deverá adotar apenas o método **doPost**. O parâmetro **acao**, indicando a solicitação de conexão ou desconexão, é obrigatório, sendo gerada uma exceção caso ele não seja fornecido.

A partir de uma instrução **switch**, com base no parâmetro **acao**, temos a implementação da autenticação e da desconexão do sistema. Observe que o Java permite uso de texto para os desvios de fluxo do switch, mas apenas nas versões atuais da plataforma.

Para responder à ação "**conectar**" é feito um teste, em que apenas o login "**admin**" e a senha com valor "**123**" permitirão a autenticação. Fornecidos os valores corretos, temos o acréscimo do atributo **usuario** à sessão, contendo o valor "**Administrador**", e ocorre o redirecionamento para a página **Segura.jsp**.



Atenção

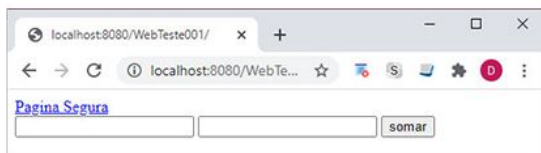
Caso sejam fornecidas credenciais diferentes das estipuladas, será definido o atributo erro para a requisição, com a mensagem "Dados Inválidos", e ocorrerá o retorno para Login.jsp, por meio de um redirecionamento interno.

Quanto à resposta para a ação "desconectar", temos um processo bem mais simples, com a chamada para o método invalidate, do objeto session, e o redirecionamento para a página index.html. Com a utilização do método invalidate, a sessão atual é fechada, e os atributos associados à conexão do usuário são removidos da memória do servidor.

Agora basta adicionar uma chamada para a página Segura.jsp em index.html.

```
<a href="Segura.jsp">Pagina Segura</a>
```

As imagens a seguir apresentam o fluxo de execução do aplicativo, que normalmente ocorreria em uma tentativa de acesso à página segura.



Página Segura

Aplicativo com link para acesso a página segura.



Acesso ao sistema

Página de login.



Esta é uma página protegida

Login bem-sucedido na página protegida.

Embora seja um processo de autenticação muito simples, com valores prefixados, você pode alterá-lo facilmente para utilizar uma base de dados e senhas criptografadas.

Verificando o aprendizado

Questão 1

Nas primeiras versões do servidor Tomcat, executávamos o mapeamento de Servlets através de arquivos XML. No entanto, as versões atuais de aplicativos Java Web usam a anotação `WebServlet` para efetuar o mapeamento. Qual atributo da anotação define o endereço Web para acessar o Servlet?

A

name

B

class

C

urlPatterns

D

link

E

Location



A alternativa C está correta.

Por meio do atributo `urlPatterns`, é definido o endereço, ou conjunto de endereços, através de coringas, a partir de onde o fluxo de execução será direcionado para os métodos de resposta do Servlet.

Questão 2

A tecnologia de páginas JSP permite mesclar HTML e Java em meio ao mesmo código, constituindo uma opção particularmente interessante para a definição do design. Qual o nome utilizado para os trechos de código escritos em Java nas páginas?

A

Diretiva

B

Expression

C

Comentário

D

Scriptlet

E

Componente



A alternativa D está correta.

Os trechos de código escritos em Java são chamados de Scriptlets, e os conteúdos são transcritos diretamente para o Servlet gerado pelo contêiner, diferentemente da parte HTML, que é transformada em instruções do tipo out.print.

O conceito de middleware

Middleware

Neste vídeo, vamos aprender sobre middleware.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Para definirmos middleware, devemos entender os conceitos de:

Front-end

É a camada de software responsável pelo interfaceamento do sistema, com o uso de uma linguagem de programação. Aqui temos os aplicativos Java Web como opção de front-end.

Back-end

Compreende o conjunto de tecnologias que pode ser acessado a partir de nosso front-end, mas que não pertence ao mesmo ambiente, como os bancos de dados e as mensagerias. Para nossos exemplos, iremos adotar o **banco de dados** Derby como back-end, com a consulta e manipulação de dados ocorrendo através do envio de comandos `SQL`, a partir do front-end.

Vejamos um exemplo:



Exemplo

As mensagerias são outro bom exemplo de back-end, com uma arquitetura voltada para a comunicação assíncrona entre sistemas, efetuada por meio da troca de mensagens. Essa é uma tecnologia crucial para diversos sistemas corporativos, como os da rede bancária.

Um grande problema, enfrentado pelas linguagens de programação mais antigas, é que deveríamos ter versões específicas do programa para acesso a cada tipo de servidor de banco de dados, como **Oracle**, **Informix**, **DB2** e **SQL Server**, entre diversos outros, o que também ocorria com relação aos sistemas de mensagerias, como, por exemplo, **MQ Series**, **JBoss MQ**, **Active MQ** e **Microsoft MQ**.

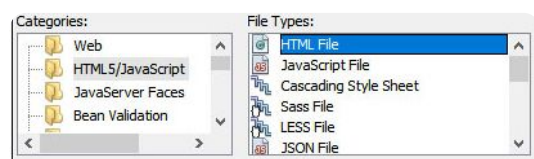


Comentário

Com diferentes componentes para acesso e modelos de programação heterogêneos, a probabilidade de ocorrência de erros é simplesmente enorme, levando à necessidade de uma camada de software intermediária, responsável por promover a comunicação entre o front-end e o back-end. Foi definido o termo **middleware** para a classificação desse tipo de tecnologia, que permite integração de forma transparente e mudança de fornecedor com pouca ou nenhuma alteração de código.

Podemos dizer que o **JDBC** (Java Database Connectivity) é o *middleware* do ambiente Java para acesso a bancos de dados. Ele permite que utilizemos produtos de diversos fornecedores, sem modificações no código do aplicativo, desde que os bancos de dados aceitem o uso de **SQL ANSI**.

Entre as diversas opções de repositórios existentes, temos o **Derby**, ou **Java DB**, um banco de dados relacional construído totalmente com tecnologia Java, que não depende de um servidor e faz parte da distribuição padrão do **JDK**. Apache Derby é um subprojeto do Apache DB, disponível sob licença Apache, e que pode ser embutido em programas Java, bem como utilizado para transações on-line.

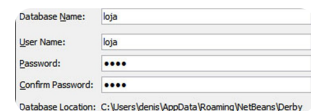


Lista de repositórios

Podemos gerenciar nossos **bancos de dados Derby** de forma muito simples, através da aba **Services** do **NetBeans**, na divisão Databases.

Passo 1

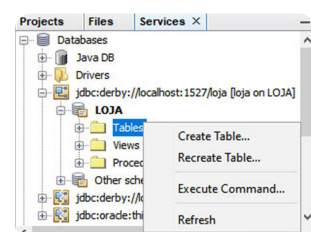
Para criarmos um banco de dados, precisamos clicar com o botão direito sobre o driver **Java DB**, selecionável com a abertura da árvore de **Databases**, seguido da escolha da opção **Create Database**, no menu de contexto. Na janela que será aberta, efetuaremos o preenchimento do nome de nosso novo banco de dados com o valor "**loja**", bem como usuário e senha, podendo também utilizar o valor "**loja**" para ambos.



Passo 2

Ao clicar no botão de confirmação, o banco de dados será criado e ficará disponível para conexão, através do driver **JDBC**. A conexão é identificada por sua **Connection String**, tendo como base o endereço de rede (**localhost**), a porta padrão (**1527**) e a instância que será utilizada (**loja**).

A conexão é aberta com o duplo clique sobre o identificador, ou o clique com o botão direito e a escolha da opção **Connect**. Com o banco de dados aberto, podemos criar uma tabela, navegando até a divisão **Tables**, no esquema **LOJA**, e utilizando o clique com o botão direito para acessar a opção **Create Table** no menu de contexto.



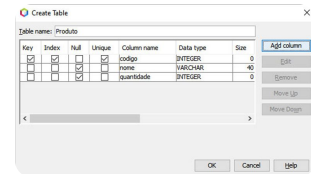
Passo 3

Na janela que se abrirá, iremos configurar uma tabela de nome **Produto**, com os campos definidos de acordo com o quadro seguinte.

Coluna	Tipo	Quantidade
codigo	INTEGER	0
nome	VARCHAR	40
quantidade	INTEGER	0

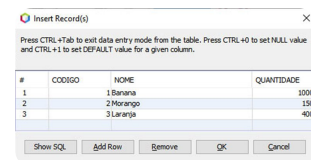
Passo 4

Definindo o nome da tabela e adicionando os campos, teremos a configuração que pode ser observada a seguir, com o processo sendo finalizado através do clique em **OK**. Cada campo deve ser adicionado individualmente, com o clique em **Add Column**.



Passo 5

A tabela criada será acessada através de um novo nó, na árvore de navegação, abaixo de **Tables**, com o nome **Produto**. Utilizando o clique com o botão direito sobre o novo nó e escolhendo a opção **View Data**, teremos a abertura de uma área para execução de **SQL** e visualização de dados no editor de código, sendo possível acrescentar registros de forma visual com o uso de **ALT+I**, ou clique sobre o ícone referente. Nesta tela de inserção, podemos preencher os valores do novo registro, e se quisermos mais de um registro, basta clicar em **Add Row**. Ao final do preenchimento dos dados, clicamos em **OK** e o NetBeans executará os comandos **INSERT** necessários.



Acesso ao banco de dados no Java

Assista ao vídeo abaixo para entender o acesso ao banco de dados no Java.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Com relação à codificação Java, os componentes do **JDBC** estão presentes no pacote **java.sql**, e o processo para utilização segue quatro passos simples:

- Instanciar a classe do driver de conexão.
- Obter uma conexão (**Connection**) a partir da Connection String, usuário e senha.
- Instanciar um executor de SQL (**Statement**).
- Executar os comandos **DML** (linguagem de manipulação de dados).

Para o comando de seleção existe mais um detalhe, que seria a recepção da consulta em um **ResultSet**, o que pode ser observado no trecho de código seguinte, no qual, com base na tabela criada anteriormente, efetuamos uma consulta aos dados inseridos.

```

java

ArrayList listaDescricoes = new ArrayList<>();

// passo 1
Class.forName("org.apache.derby.jdbc.ClientDriver");

// passo 2
Connection c1 = DriverManager.getConnection(
    "jdbc:derby://localhost:1527/loja",
    "loja", "loja");

// passo 3
Statement st = c1.createStatement();

// passo 4 e recepção no ResultSet
ResultSet r1 = st.executeQuery("SELECT * FROM PRODUTO");

while(r1.next())
    listaDescricoes.add("Produto " + r1.getInt("codigo")+
        ": " + r1.getString("nome"));

r1.close();
st.close();
c1.close();

```

No início do código, temos o driver Derby sendo instanciado a partir de uma chamada para o método **forName**. Com o driver instanciado, ele fica disponível para o aplicativo, permitindo abrir as conexões com o banco de dados através de **JDBC**.

Em seguida, é instanciada a conexão **c1**, por meio da chamada ao método **getConnection**, da classe **DriverManager**, e os parâmetros fornecidos são a **Connection String**, o **usuário** e a **senha**. Quanto à Connection String, ela pode variar muito, sendo iniciada pelo driver utilizado, seguido dos parâmetros específicos para aquele driver, que, no caso do Derby, são o endereço de rede, a porta e o nome do banco de dados.

A partir da conexão **c1**, é gerado um executor de SQL de nome **st**, com a chamada para o método **createStatement**. Com o executor instanciado, realizamos uma consulta ao banco, através da invocação do método **executeQuery**, recebendo um **ResultSet**.



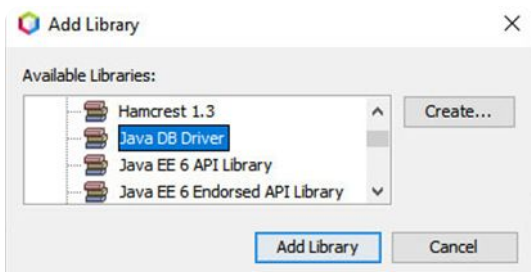
Atenção

As consultas ao banco são feitas com a utilização de **executeQuery**, mas os comandos para manipulação de dados são executados através de **executeUpdate**.

Após a recepção da consulta no objeto de nome **r1**, podemos nos movimentar pelos registros, com o uso de **next**, e acessar cada campo pelo nome para a obtenção do valor, sempre lembrando de utilizar o método correto para o tipo do campo, como **getString** para texto e **getInt** para valores numéricos inteiros.

Ao efetuar a consulta, o **ResultSet** fica posicionado antes do primeiro registro, na posição **BOF** (Beginning of File), e com o uso do comando **next** podemos mover para as posições seguintes, até atingir o final da consulta, na posição **EOF** (End of File). A cada registro visitado, adicionamos a descrição do produto ao **ArrayList**.

Na parte final, devemos fechar os componentes JDBC, na ordem inversa daquela em que foram criados, já que existe dependência sucessiva entre eles.



Biblioteca

Caso a biblioteca JDBC, para o tipo de banco de dados escolhido não esteja presente no servidor, devemos adicioná-la ao projeto, ou ocorrerá erro durante a execução.

Para o nosso exemplo, podemos adicionar a biblioteca **Java DB Driver** clicando com o botão direito na divisão **Libraries** e usando a opção **Add Library**.

DAO e aplicativos web

Demonstração de uso de componentes JDBC

Assista ao vídeo a seguir para saber mais sobre os aplicativos web com uso de DAO.



Conteúdo interativo

Acesse a versão digital para assistir ao vídeo.

Data access object (DAO)

Agora que sabemos efetuar as operações sobre o banco de dados, seria interessante organizar a forma de programar, pois é fácil imaginar a dificuldade de manutenção em um sistema com dezenas de milhares de linhas de código Java, contendo diversos comandos SQL espalhados ao longo das linhas.

A **linguagem Java** é orientada a objetos, e torna-se mais fácil representar uma tabela através de uma classe, onde suas instâncias corresponderão aos registros existentes.

```
java

public class Produto {
    public int codigo;
    public String nome;
    public int quantidade;

    public Produto(){ }

    public Produto(int codigo, String nome, int quantidade) {
        this.codigo = codigo;
        this.nome = nome;
        this.quantidade = quantidade;
    }
}
```

Com base na observação de que o SQL espalhado ao longo do código traz grandes dificuldades, foi desenvolvido o padrão DAO, com o objetivo de concentrar as instruções SQL em um único tipo de classe, o que permite o agrupamento e a reutilização dos diversos comandos relacionados ao banco de dados. Normalmente, temos uma classe DAO para cada classe de entidade relevante para o sistema.

```

java

public class ProdutoDAO {
    private Connection getConnection() throws Exception{
        Class.forName("org.apache.derby.jdbc.ClientDriver");
        return DriverManager.getConnection(
            "jdbc:derby://localhost:1527/loja",
            "loja", "loja");
    }
    private Statement getStatement() throws Exception{
        return getConnection().createStatement();
    }
    private void closeStatement(Statement st) throws Exception{
        st.getConnection().close();
    }

    public List obterTodos(){
        ArrayList lista = new ArrayList<>();
        try {
            ResultSet r1 = getStatement().executeQuery(
                "SELECT * FROM PRODUTO");
            while(r1.next())
                lista.add(new Produto(r1.getInt("codigo"),
                    r1.getString("nome"),r1.getInt("quantidade")));
            closeStatement(r1.getStatement());
        }catch(Exception e){
        }
        return lista;
    }
}

```

Inicialmente, criamos os métodos **getStatement** e **closeStatement** com o objetivo de gerar executores de SQL e eliminá-los, efetuando também as conexões e desconexões nos momentos necessários. Outro método é o **getConnection**, utilizado apenas para encapsular o processo de conexão com o banco.



Atenção

O método **obterTodos** irá retornar todos os registros da tabela **Produto** no formato de um **ArrayList** de entidades do tipo **Produto**. Primeiramente, é executado o SQL necessário para a consulta e, para cada registro obtido no cursor, é gerado um novo objeto da classe **Produto** e adicionado à lista de retorno.

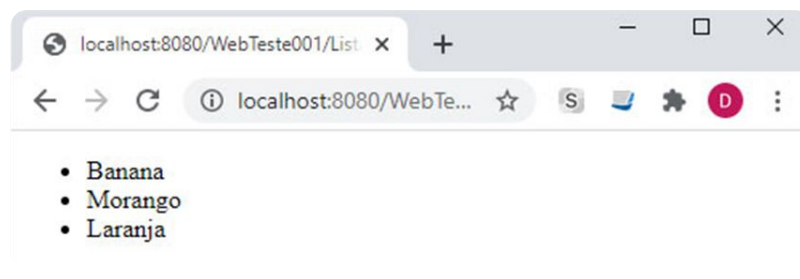
Podemos alterar o código de **ListaCores.jsp** para exibir o nome de cada produto.

plain-text

```
<%@page import="model.Produto"%>
<%@page import="model.ProdutoDAO"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<%
    ProdutoDAO dao = new ProdutoDAO();
    for (Produto p : dao.obterTodos()) {
        out.println("
• " + p.nome + "
");
    }
%>
```

A princípio, foi necessário importar as classes **Produto** e **ProdutoDAO**, com a utilização da diretiva **page**, viabilizando o uso delas no **Scriptlet**. A partir daí, bastou instanciar o **DAO**, efetuar a chamada para **obterTodos** e percorrer a coleção, acrescentando os itens de lista para cada produto recebido.



Agora que testamos o acesso ao banco de dados no ambiente Web, vamos acrescentar os métodos para efetuar a **inclusão** e a **exclusão** de produtos na classe **ProdutoDAO**.

```
java

public void excluir(int codigo){
    try {
        Statement st = getStatement();
        st.executeUpdate(
            "DELETE FROM PRODUTO WHERE CODIGO = "+
            codigo);
        closeStatement(st);
    }catch(Exception e){
    }
}
```

Podemos observar que a exclusão utiliza um **Statement**, apenas com a observação de que em vez de utilizar **executeQuery**, como nas seleções de dados, adotamos o método **executeUpdate**, por se tratar de um comando DML. Com base no código fornecido, um comando SQL é montado, visando à exclusão do produto identificado pelo código.

```

java
public void incluir(Produto p){
    try {
        PreparedStatement ps = getConnection().prepareStatement(
            "INSERT INTO PRODUTO VALUES(?,?,?)");
        ps.setInt    (1, p.codigo);
        ps.setString(2, p.nome);
        ps.setInt    (3, p.quantidade);
        ps.executeUpdate();
        closeStatement(ps);
    }catch(Exception e){
    }
}
}

```

Na inclusão, foi utilizado um elemento do tipo **PreparedStatement**, que permite a construção de comandos SQL parametrizados. O uso de parâmetros facilita a escrita do comando SQL, sem a preocupação com o uso de apóstrofe ou outro delimitador, sendo particularmente útil quando tivermos de trabalhar com datas.



Dica

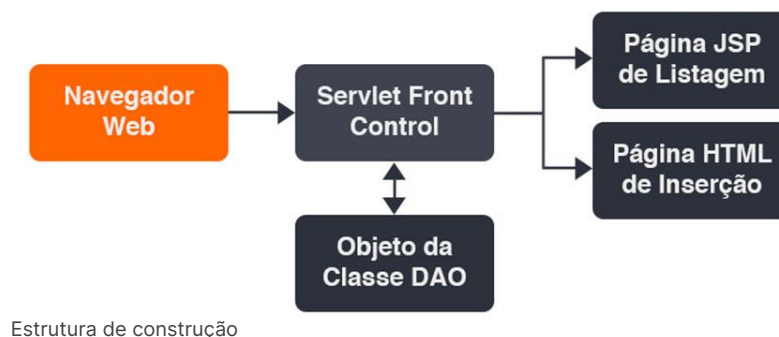
Para definir os parâmetros, utilizamos pontos de interrogação, que assumem valores posicionais, a partir de um, o que é um pouco diferente da indexação dos vetores, que começa em zero.

Os parâmetros são preenchidos com a chamada ao método correto de acordo com seus tipos, como **setInt** para inteiro e **setString** para texto. Após o preenchimento, devemos executar o comando SQL, com a chamada para **executeUpdate** no caso das instruções **INSERT**, **UPDATE** e **DELETE**, ou **executeQuery** para a instrução **SELECT**.

Com a codificação completa para as classes Produto e ProdutoDAO, estamos preparados para implementar nossa interface cadastral com o uso de **Servlets** e **JSPs**.

Aplicativos web com uso de DAO

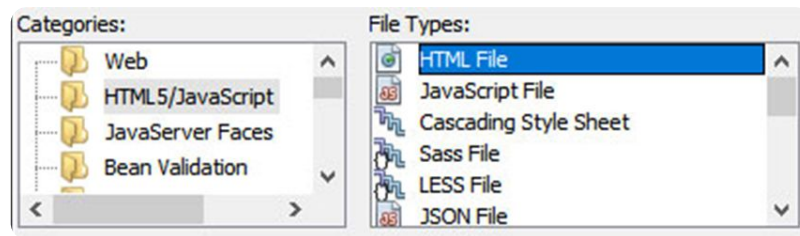
A construção de nosso aplicativo irá se concentrar na melhor utilização de **Servlets** e **JSPs**, ou seja, teremos o **Servlet** como um conversor de formatos e redirecionador de fluxo, enquanto o **JSP** será responsável pela construção da interface com o usuário.



Nosso sistema seguirá o padrão de desenvolvimento **Front Controller**, em que todas as requisições serão recebidas no Servlet, onde ocorrerão as conversões e os processamentos necessários, com o redirecionamento para a visualização correta ao final. Tendo como base o comportamento descrito, fica clara

a necessidade de um parâmetro que permita diferenciar qual fluxo deverá ser executado, ao qual daremos o nome de **"acao"**.

Vamos começar pela parte mais simples, acrescentando um arquivo do tipo **HTML File**, na categoria **HTML5/JavaScript**, com o nome **ProdutoDados**.



Nossa página **HTML** será constituída apenas de um formulário, com campos de entrada para os atributos de **Produto**, além de um campo escondido (**hidden**) para definir o valor de **acao** como **"inserirX"**. Após o usuário preencher os dados e clicar no botão **incluir**, ocorrerá o envio pelo método **post** para o **Servlet**.

plain-text

Codigo:

Nome:

Quantidade:

Em seguida, vamos adicionar um arquivo do tipo **JSP**, na categoria **Web**, com o nome **ProdutoLista**. A nova página será responsável pela exibição dos dados emitidos a partir do Servlet, além de apresentar links para inclusão e exclusão de produtos.

plain-text

```
<%@page import="model.Produto"%>
<%@page import="java.util.ArrayList"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

[Incluir Produto](#)

```
<%
    ArrayList lista = (ArrayList)
        request.getAttribute("listaProduto");
    for(Produto p: lista) {
%>

<% } %>
```

codigo	nome	quantidade	Opções
<%=p.codigo%>	<%=p.nome%>	<%=p.quantidade%>	excluir

Na parte inicial da página, temos um *link* para a inclusão de produto, com a chamada para o Servlet, tendo o parâmetro **acao** com valor "**incluir**". No nível do Servlet, o acesso corresponderá a um redirecionamento direto para a página **HTML** de cadastro.

Em seguida, temos a criação de uma tabela para a exibição dos dados de cada produto, a partir de uma lista fornecida em um atributo da requisição. A tabela apresenta uma linha inicial com os nomes dos campos, e as demais linhas, com os valores, são criadas dinamicamente através do código do **Scriptlet**.



Atenção

Como os atributos podem ser de qualquer tipo de objeto, é necessário efetuar o type cast ao recebê-los, via método `getAttribute`.

Após resgatar a lista, executamos um **loop** do tipo **for each** para visitar todos os produtos da lista, e para cada um é executado o bloco de repetição. Note que o bloco, delimitado com o uso de chaves, contém elementos **HTML** e **Scriptlets**, uma técnica muito interessante, que nos permite aproveitar a melhor forma de escrita a cada ponto.

Dentro do bloco, temos a construção de uma linha da tabela HTML, onde as colunas são preenchidas com o valor dos campos por meio de etiquetas de substituição, demarcadas com o uso dos símbolos de percentual e igual. Em termos práticos, a utilização de uma etiqueta de substituição equivale a um **Scriptlet** com o comando **out.print**.

plain-text

```
<!-- Formas Equivalentes --%>
<%= p.codigo %>
<% out.print(p.codigo); %>
```

Podemos observar como a combinação entre **HTML** e **etiquetas de substituição** é muito fluida, principalmente na montagem do link, em que cada linha terá um código diferente, fazendo referência ao produto corrente. O link de exclusão utiliza o parâmetro **acao** com valor **"excluirX"**, enquanto o parâmetro **codigo** recebe o valor dinamicamente.

Agora chegou o momento de criar o **Servlet**, orquestrando a funcionalidade de nossas páginas e executando os processos necessários para cada chamada. Vamos adicionar um Servlet com o nome **ServletProduto**.

plain-text

```
@WebServlet(name = "ServletProduto",
            urlPatterns = {"/ServletProduto"})
public class ServletProduto extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
                                  HttpServletResponse response)
        throws ServletException, IOException {
        String acao = request.getParameter("acao");
        if(acao==null) acao="listar";
        String paginaDestino = (acao.equals("incluir"))?
            "ProdutoDados.html" : "ProdutoLista.jsp";
        ProdutoDAO dao = new ProdutoDAO();
        switch(acao){
            case "incluirX":
                int codigo = new Integer(request.getParameter("codigo"));
                String nome = request.getParameter("nome");
                int quantidade = new Integer(
                    request.getParameter("quantidade"));
                dao.incluir(new Produto(codigo,nome,quantidade));
                break;
            case "excluirX":
                dao.excluir(new Integer(request.getParameter("codigo")));
                break;
        }
        if(!acao.equals("incluir")){
            request.setAttribute("listaProduto", dao.obterTodos());
        }
        request.getRequestDispatcher(paginaDestino).
            forward(request, response);
    }

    // O restante do código foi omitido
}
```

O primeiro passo, na execução do Servlet, é a captura do parâmetro **acao**, e se ele não é fornecido, assumimos o valor padrão **"listar"**. Com base nesse parâmetro, teremos a escolha da página de destino, que será **ProdutoDados.html** para o início do processo de inclusão e **ProdutoLista.jsp** para as demais ações.

Em seguida, é instanciado um objeto do tipo **ProdutoDAO**, com o nome **dao**, para dar suporte aos processos que devem ser iniciados nas ações.

Para **acao** com o valor **"incluirX"**, capturamos os valores fornecidos via **HTTP**, efetuando a conversão para inteiro, quando necessário, e invocamos o método **incluir**, do objeto **dao**, com a passagem de uma instância de

Produto, inicializada com os valores obtidos. Para o valor "**excluirX**", resgatamos o valor do parâmetro **codigo**, convertendo para inteiro, e efetuamos uma chamada para o método **excluir**.



Comentário

A finalização do processo de inclusão, após o preenchimento dos dados e envio pelo usuário, bem como a exclusão de um registro e a simples listagem, direciona para a página ProdutoLista.jsp, onde deve ser resgatado o atributo listaProduto. Com base na necessidade, temos a chamada para setAttribute, no nível do Servlet, para qualquer ação que seja diferente do início da inclusão.

Ao final, temos o redirecionamento para a página correta, com o método **forward**, e a construção da resposta ficará a cargo dela, completando o ciclo de resposta.

Agora só precisamos efetuar a chamada correta para o Servlet.

A tela de listagem será apresentada, fornecendo também os *links* necessários para efetuar a inclusão e exclusão de produtos.



codigo	nome	quantidade	Opções
1	Banana	1000	excluir
2	Morango	150	excluir
3	Laranja	400	excluir

Tela de listagem

Verificando o aprendizado

Questão 1

De modo geral, quando criamos uma classe DAO, temos a necessidade de criar os comandos SQL de inserção, alteração ou exclusão de forma dinâmica, tendo como base os atributos presentes nas entidades. Para evitar uma grande quantidade de conversões e concatenações ao longo do SQL, podemos trabalhar com uma classe do tipo:

A

PreparedStatement

B

ResultSet

C

Statement

D

Connection

E

Exception



A alternativa A está correta.

A classe PreparedStatement permite definir comandos SQL parametrizados, sendo os parâmetros definidos de forma posicional, através de interrogações, e os valores preenchidos já no formato correto, com base em métodos como setInt e setString, evitando a necessidade de conversões manuais ou concatenações.

Questão 2

Um padrão de desenvolvimento muito utilizado na construção de sistemas Web é a concentração das chamadas em um ponto único, onde são feitas as conversões necessárias, a execução de métodos de negócios e o redirecionamento para a visualização correta. Qual o nome desse padrão de desenvolvimento?

A

Session Facade

B

DAO

C

Flyweight

D

Front Controller

E

Composite



A alternativa D está correta.

No ambiente Java Web, é comum a utilização do padrão Front Controller, que tem o objetivo de centralizar as chamadas efetuadas para o sistema, sendo frequente a utilização de Servlets na implementação.

Considerações finais

Neste estudo, analisamos dois servidores de grande relevância para a plataforma Java: o **Web Server** Tomcat e o **Application Server** GlassFish. Observamos como eles podem ser configurados e utilizados, além de compreender o formato correto para a geração de aplicativos para os servidores.

Vimos que as tecnologias Servlet e JSP oferecem o suporte necessário para responder às solicitações do protocolo HTTP, além de permitirem a gerência de sessões, uma solução ao nível do servidor para a gerência de estados.

Finalmente, entendemos o funcionamento do **middleware JDBC**, permitindo a criação de **sistema Java Web** com acesso a bancos de dados, implementando um exemplo que serve de base para a construção de sistemas cadastrais baseados em **Servlets** e **JSPs**.

Podcast

Agora, encerramos nosso estudo abordando um pouco mais sobre programação Servidor com Java.



Conteúdo interativo

Acesse a versão digital para ouvir o áudio.

Explore +

Para se aprofundar mais neste tema, sugerimos as seguintes leituras:

- Manual do Application Server GlassFish, de 2017.
- Tutorial de Servlet e JSP, apresentado pelo Journal Dev.
- Introdução ao Desenvolvimento de Aplicações Web, disponível na página do NetBeans.
- O que é JDBC: Drivers, Connections e consultas SQL, de Marco Behler.

Referências

CASSATI, J. P. **Programação servidor em sistemas web**. Rio de Janeiro: Estácio, 2016.

CORNELL, G; HORSTMANN, C. **Core Java**. 8 ed. São Paulo: Pearson, 2010.

DEITEL, P; DEITEL, H. AJAX. **Rich Internet Applications e Desenvolvimento Web para Programadores**. São Paulo: Pearson Education, 2009.

DEITEL, P; DEITEL, H. JAVA. **Como programar**. 8 ed. São Paulo: Pearson, 2010.

FONSECA, E. **Desenvolvimento de Software**. Rio de Janeiro: Estácio, 2015.

MONSON-HAEFEL, R; BURKE, B. **Enterprise Java Beans 3.0**. 5th ed. USA: O'Reilly, 2006.

SANTOS, F. **Programação I**. Rio de Janeiro: Estácio, 2017.